Road Accident Severity Prediction

Machine Learning Project Report

◆◆◆

Name: Devanshu

Course: B.Tech CSE (AI & ML), 2nd Year

College: Geeta University, Panipat

Internship: Samatrix.io

1. Introduction

This report is about my Road Accident Severity Prediction project. The idea of the project is to build a Machine Learning model that looks at the details of a road accident (weather, road type, lighting, number of vehicles, etc.) and predicts how serious the accident is likely to be. There are three possible outputs — Slight Injury, Serious Injury, or Fatal Injury — so this is a Multi-Class Classification problem.

The dataset is real accident data collected by the Addis Ababa Sub-city Police Department (Ethiopia) between 2017 and 2020, with 12,316 records and 32 columns. I have written the full code in a Jupyter Notebook (RTA_PROJECT.ipynb). In this report I am answering all the 'Check Your Understanding' questions from the project sheet, step by step, using the actual results I got while running my notebook.

STEP 1: Set Up the Python Environment

In this step, the required Python libraries were imported: pandas, numpy, matplotlib, seaborn, and scikit-learn (for LabelEncoder, train_test_split, LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, and evaluation metrics).

Q1. What is the purpose of "importing" a library in Python? What happens if you try to use a library's function without importing it first?

Importing a library means telling Python to load a set of ready-made tools (functions and classes) so we can use them in our code without writing them from scratch. If we try to use a library's function without importing it first, Python will give a NameError, saying that the function or library name is not defined.

Q2. In your own words, explain the difference between pandas and numpy.

Pandas is mainly used to work with tabular data, meaning data arranged in rows and columns, like an Excel sheet. It is great for loading CSV files, cleaning data, and analysing columns. Numpy is used for fast numerical and array calculations, like doing maths on large sets of numbers. Pandas is actually built on top of numpy, so they work well together.

Q3. Which library will you use to draw a bar chart or a heatmap in this project?

Matplotlib and seaborn were used to draw the charts in this project. Matplotlib was used for basic plots like the accuracy comparison bar chart, and seaborn was used for the count plots, the missing values bar chart, and the confusion matrix heatmap.

Q4. Name at least two things scikit-learn provides that you will need for this project.

Scikit-learn provided the Machine Learning models used in this project (Logistic Regression, Decision Tree, and Random Forest), the train_test_split function to split the data into training and testing sets, the LabelEncoder to convert text columns into numbers, and evaluation tools like accuracy_score, confusion_matrix, and classification_report.

STEP 2: Load the Dataset

The dataset was loaded using pandas' read_csv() function, and the first few rows were displayed using df.head() to confirm it loaded correctly.

Q1. What is a CSV file? Why is it a common format for sharing datasets?

A CSV (Comma Separated Values) file is a simple text file where each line is one row of data, and the values in that row are separated by commas. It is a common format for sharing data because it is

lightweight, easy to read, and can be opened by almost any tool, including Excel, pandas, and databases.

Q2. What is a pandas DataFrame? How would you describe it to someone who has only used Excel before?

A pandas DataFrame is a table-like data structure with rows and columns, very similar to a sheet in Excel. Each column can hold a different type of data (text, numbers, dates), and we can filter, sort, group, or clean the data using simple pandas commands, just like using formulas and filters in Excel.

Q3. Why is it important to preview the first few rows of data immediately after loading it, instead of moving straight to the next step?

Previewing the first few rows confirms that the data has loaded correctly, the columns look as expected, and there are no obvious formatting problems. It also gives a first look at what kind of values are present in each column before doing any deeper analysis.

Q4. If the dataset link was broken, what error or output do you think you would see?

If the dataset link was broken, pandas would raise an error while trying to fetch or parse the file, such as a connection error, a 404 (file not found) error, or a parsing error if it received an empty or invalid response instead of a proper CSV file.

STEP 3: Explore the Dataset (EDA)

The dataset shape, column info, target class counts, and missing values were checked.

Key results from the notebook:

- Shape of dataset: 12,316 rows and 32 columns.
- Target column counts (Accident_severity): Slight Injury = 10,415, Serious Injury = 1,743, Fatal Injury = 158.
- Columns with the highest number of missing values: Defect_of_vehicle (4,427 missing) and Service_year_of_vehicle (3,928 missing), followed by Work_of_casuality (3,198) and Fitness_of_casuality (2,635).

Q1. Why should you explore a dataset before jumping straight into building a model?

Exploring the dataset first helps us understand what kind of data we are working with — how many rows and columns it has, which columns have missing values, and how balanced the target classes are. Without this understanding, we might build a model on messy or misleading data and get poor or wrong results.

Q2. What did you notice about the number of Slight Injury accidents compared to Fatal Injury accidents? What is this situation called in Machine Learning, and why can it be a problem?

Slight Injury accidents (10,415) are far more common than Fatal Injury accidents (only 158). This is called Class Imbalance in Machine Learning. It is a problem because the model can become biased towards predicting the majority class (Slight Injury) most of the time, since that gives high accuracy overall, while it may fail to correctly identify the rare but important Fatal Injury cases.

Q3. Name two columns that had the highest number of missing values. Why do you think real police accident records might have missing information for these columns?

Defect_of_vehicle and Service_year_of_vehicle had the highest number of missing values. This is likely because, at an accident scene, police officers may not always be able to check or verify these vehicle details on the spot, especially if the vehicle is damaged, the owner is not present, or the paperwork is unavailable at that time.

Q4. What is the difference between a categorical column and a numerical column? Give one example of each from this dataset.

A categorical column contains text-based labels or categories, such as Weather_conditions (Normal, Raining, Windy). A numerical column contains numbers that can be measured or counted, such as Number_of_casualties. Categorical columns need to be encoded into numbers before a Machine Learning model can use them, while numerical columns can usually be used directly.

Q5. What did the bar chart of accident severity visually tell you that a plain list of numbers did not?

The bar chart made the huge gap between the three classes immediately visible — the Slight Injury bar was much taller than the other two. While the plain numbers also show this, the chart makes the size of the imbalance much easier and faster to understand at a glance.

STEP 4: Clean the Dataset (Handle Missing Values)

All categorical (text) columns with missing values were filled with the text "Unknown", and all numerical columns with missing values were filled with the median of that column. After this, the total number of missing values across the whole dataset became 0.

Q1. Why can't a Machine Learning algorithm work directly with missing (NaN) values?

Machine Learning algorithms perform mathematical calculations on the data. A missing (NaN) value has no numeric meaning, so the algorithm cannot calculate distances, probabilities, or splits using it, and most scikit-learn models will throw an error if NaN values are present.

Q2. Why did we choose to fill missing values instead of simply deleting every row that has at least one missing value?

Many columns had a large number of missing values (for example, over 4,000 missing in Defect_of_vehicle). If we deleted every row with at least one missing value, we would lose a very large portion of the 12,316 records, which would leave us with much less data to train a good model.

Q3. What is a median? Why might it be a safer choice than the average (mean) for filling missing numeric values, especially if some values are unusually large or small?

The median is the middle value of a column when all the values are sorted from smallest to largest. It is a safer choice than the mean because the mean can be pulled up or down a lot by unusually large or small values (outliers), while the median stays close to where most of the data actually is.

Q4. This process of filling in missing data is called "Imputation". Can you think of one disadvantage or risk of filling missing values instead of collecting the real data?

The main risk is that imputed values are guesses, not real facts. Filling categorical values with 'Unknown' or numeric values with the median can slightly change the true pattern in the data, and if a column has too many missing values, the filled-in values might not reflect reality well, which can reduce the accuracy of the model.

STEP 5: Feature Engineering and Feature Selection

Part A: The Time column (like 17:02:00) was converted into a new Hour column (like 17), and the original Time column was removed.

Part B: The following 'after-the-fact' casualty columns were removed because they describe the victim after the accident and would cause data leakage: Casualty_class, Sex_of_casualty, Age_band_of_casualty, Casualty_severity, Fitness_of_casuality, and Pedestrian_movement. After this, the final feature set (X) had 25 columns and 12,316 rows, and the target (y) had 12,316 values.

Q1. What is Feature Engineering? Why is converting the exact Time into an Hour value more useful for the model?

Feature Engineering means creating new, more useful columns from the existing raw data. Converting the exact Time (like 17:02:00) into just the Hour (like 17) is more useful because the model can now find patterns based on the hour of the day, such as more accidents during evening traffic or fewer during late night, instead of treating each exact second as a completely different value.

Q2. Why did we decide to exclude "after-the-fact" casualty columns from our feature list?

Columns like Casualty_class, Sex_of_casualty, and Fitness_of_casuality describe information about the victim that is only known after the accident has already happened and its severity has already been decided. Since these would not be available at the time we actually need to make a prediction, they were removed from the feature list.

Q3. What could go wrong with our model's real-world usefulness if we included information that is only known after the accident's severity is already determined? (This idea is called "data leakage" — try to explain it in your own words.)

This is called data leakage. If we include information that is only known after the outcome has already happened, the model can use it to 'cheat' and get very high accuracy during testing, but it will fail badly in the real world, because at the moment we actually need a prediction (right after the accident occurs), that after-the-fact information simply will not be available yet.

Q4. Can you think of one more new feature you could create by combining two existing columns (for example Day_of_week and Hour)? What might it help the model learn?

We could combine Day_of_week and Hour to create a feature like 'Weekend_Night', which is True if the day is Saturday or Sunday and the hour is late at night. This could help the model learn that weekend nights may have a different accident severity pattern compared to regular weekday hours, for example due to different traffic or driving behaviour.

STEP 6: Encode Categorical Columns

All categorical feature columns were converted into numbers using scikit-learn's LabelEncoder, one column at a time. A separate LabelEncoder was used for the target column (Accident_severity). After encoding, the mapping obtained was:

- Fatal injury --> 0
- Serious Injury --> 1
- Slight Injury --> 2

Q1. Why can't Machine Learning algorithms directly understand text categories?

Machine Learning algorithms are built on mathematics — they calculate distances, probabilities, and weighted sums. Text values like 'Raining' or 'Normal' have no numeric meaning, so they must first be converted into numbers before any algorithm can process them.

Q2. In your own words, explain how Label Encoding works.

Label Encoding looks at all the unique categories in a column and assigns each one a different integer number, starting from 0. For example, in the target column, 'Fatal injury' became 0, 'Serious Injury' became 1, and 'Slight Injury' became 2. This way, the text data is turned into numbers the model can use.

Q3. Why did we encode the target column separately from the other feature columns?

The target column was encoded with its own separate LabelEncoder so that its specific number-to-label mapping (0 = Fatal injury, 1 = Serious Injury, 2 = Slight Injury) could be tracked and reversed later using inverse_transform(), without mixing it up with the encoders used for the feature columns.

Q4. Label Encoding assigns numbers like 0, 1, 2 to categories. Do you think this could confuse a model into thinking one category is "greater than" another, when the categories actually have no natural order (for example, weather types)? Look up the term "One-Hot Encoding" — how is it different?

Yes, this can be a problem. For a column like Weather_conditions, Label Encoding might turn 'Normal' into 0, 'Raining' into 1, and 'Windy' into 2. Some models (especially Logistic Regression) might wrongly assume that 'Windy' (2) is somehow bigger or more important than 'Normal' (0), even though weather types have no real order. One-Hot Encoding avoids this by creating a separate 0/1 column for each category instead of a single number, so there is no false sense of order, though it does create more columns.

STEP 7 & 8: Prepare Features/Target and Train-Test Split

The features (X) and target (y) were separated. The data was then split into a training set and a testing set using train_test_split, with test_size = 0.20, random_state = 42, and stratify = y. The resulting split was:

- Training Features: 9,852 rows, 25 columns | Training Target: 9,852 values
- Testing Features: 2,464 rows, 25 columns | Testing Target: 2,464 values

Class distribution in the training set: Slight Injury (2) = 8,331, Serious Injury (1) = 1,394, Fatal injury (0) = 127. Class distribution in the testing set: Slight Injury (2) = 2,084, Serious Injury (1) = 349, Fatal injury (0) = 31. Both sets keep almost the same proportion of each class, confirming that stratify worked correctly.

Q1. What is the difference between X (features) and y (target) in a Machine Learning problem?

X contains the input columns that describe an accident (like weather, road type, number of vehicles), which the model uses to make a prediction — it's the 'question'. y is the actual outcome we want to predict (Accident_severity) — it's the 'correct answer' that the model tries to learn to produce.

Q2. Why do we split data into training and testing sets instead of using all the data to train the model?

If we train and test on the exact same data, the model might just memorise the answers instead of truly learning the patterns, so it would look very accurate but actually be unreliable. Splitting the data lets us test the model on data it has never seen (2,464 test rows), which tells us how well it will actually perform on new, real-world accidents.

Q3. What does the "stratify" setting do, and why is it especially important for this dataset? (Think back to what you observed in Step 3 about class imbalance.)

The stratify=y setting makes sure that both the training set and the testing set contain roughly the same proportion of each severity class as the full dataset. This is especially important here because the classes are heavily imbalanced (very few Fatal injury cases); without stratify, the test set could end up with very few or even zero Fatal injury examples, making the evaluation unreliable.

Q4. What problem could occur if we tested our model on the same data it was trained on?

The model could simply memorise the training examples rather than genuinely learning the underlying patterns. This would make the accuracy look artificially high during testing, but the model would likely perform much worse on new, unseen accident records in the real world.

Q5. Why do we fix a "random state" (seed) value when splitting the data?

Setting random_state = 42 makes the split reproducible — meaning every time the code is run, the exact same rows go into the training set and the exact same rows go into the testing set. This makes it possible to get consistent, comparable results and to debug or share the project reliably.

STEP 9: Train Machine Learning Models

Three models were trained on the training data (X_train, y_train): Logistic Regression, Decision Tree, and Random Forest (with n_estimators = 100). All three models were created with class_weight = 'balanced' to help them pay more attention to the rare Fatal injury and Serious Injury classes.

Q1. In your own words, explain how a Decision Tree makes a prediction.

A Decision Tree works like a flowchart of yes/no questions about the features. Starting from the top, it asks a question about one feature (for example, 'Is Number_of_vehicles_involved greater than 2?'), and depending on the answer, it moves down the correct branch. It keeps asking further questions until it reaches a final leaf, which gives the predicted severity class.

Q2. Why does a Random Forest usually perform better than a single Decision Tree?

A Random Forest builds many different Decision Trees, each trained on a slightly different random sample of the data and features, and then combines their predictions by voting. This reduces the risk of one tree overfitting to noise in the data, so the combined result is usually more accurate and stable. In this project, Random Forest indeed gave the best accuracy (84.7%) compared to a single Decision Tree (77.35%).

Q3. What does the class_weight='balanced' setting do, and why is it useful for this specific dataset?

class_weight='balanced' automatically gives more importance (weight) to the minority classes (Fatal injury and Serious Injury) and less weight to the majority class (Slight Injury) during training. This is useful here because the dataset is heavily imbalanced (only 158 Fatal injury cases out of 12,316), and without this setting, the model could simply ignore the rare classes and still get a high overall accuracy.

Q4. What information does a model actually "learn from" during the training (fitting) process?

During training (the .fit() step), the model looks at the feature values (X_train) alongside their correct labels (y_train) and adjusts its internal parameters (like the splits in a Decision Tree, or the weight given to each feature in Logistic Regression) so that its predictions match the correct labels as closely as possible.

STEP 10: Evaluate the Models

Accuracy Comparison:

Model	Accuracy
Logistic Regression	49.03%
Decision Tree	77.35%
Random Forest (Best Model)	84.70%

Random Forest was selected as the best model based on accuracy. Its classification report showed strong performance on the Slight Injury class, but, like the other models, weaker precision on the rare Fatal injury class — which is expected given how few Fatal injury examples exist (only 31 in the test set) compared to Slight Injury (2,084).

Q1. Why can accuracy alone be a misleading measure of performance for this dataset? (Hint: think about the class imbalance you observed earlier.)

Since Slight Injury makes up about 85% of all records, a model could get around 85% accuracy just by predicting 'Slight Injury' for every single case, without actually learning anything useful about Serious or Fatal injuries. This is why accuracy alone is misleading here, and metrics like precision, recall, and the confusion matrix (which look at each class separately) are more meaningful.

Q2. In your own words, define Precision and Recall. Which one do you think matters more for predicting Fatal accidents, and why?

Precision answers: out of everything the model predicted as Fatal, how many were actually Fatal? Recall answers: out of all the accidents that were truly Fatal, how many did the model correctly catch? For predicting Fatal accidents, Recall matters more, because missing a real Fatal case (a false negative) can be far more dangerous in the real world than raising a false alarm — emergency services would rather over-respond than under-respond to a truly fatal accident.

Q3. What does the diagonal of a Confusion Matrix represent? What do the values outside the diagonal represent?

The diagonal of the confusion matrix represents the correct predictions — cases where the predicted class matches the actual class. The values outside the diagonal represent the mistakes, showing exactly which class the model confused with which other class (for example, a Serious Injury accident that was wrongly predicted as Slight Injury).

Q4. Which of your three models performed the best overall? Give at least one possible reason why it performed better than the others.

Random Forest performed the best overall, with 84.70% accuracy, compared to 77.35% for Decision Tree and only 49.03% for Logistic Regression. This is likely because Random Forest combines many Decision Trees together, which helps it capture complex, non-linear relationships between the features and reduces overfitting compared to a single Decision Tree, while Logistic Regression assumes simpler, more linear relationships that may not fit this data well.

Q5. If someone told you their model has 95% accuracy on this dataset, would you immediately trust it? What else would you want to check first?

No, a very high accuracy on this dataset would raise doubt rather than trust, because Slight Injury alone makes up about 85% of the data. Before trusting such a number, it would be important to check the confusion matrix and the classification report (precision, recall, and f1-score for each class), especially for the Fatal injury class, to make sure the model is not just predicting the majority class every time.

STEP 11: Find the Most Important Factors (Feature Importance)

Using the feature_importances_ attribute of the trained Random Forest model, the top features were:

- Hour (0.0954)
- Number_of_casualties (0.0854)
- Cause_of_accident (0.0699)
- Day_of_week (0.0658)
- Type_of_vehicle (0.0638)
- Driving_experience (0.0509)
- Area_accident_occured (0.0506)
- Number_of_vehicles_involved (0.0504)
- Age_band_of_driver (0.0488)
- Types_of_Junction (0.0458)

Q1. What does "Feature Importance" tell us about a trained model?

Feature Importance tells us how much each input column (feature) contributed to the Random Forest's predictions. A higher importance score means that feature was used more often, and more effectively, across the many trees to help separate the different severity classes.

Q2. Which features turned out to be the most important in your model? Does this match your own intuition about what typically affects accident severity in real life?

The most important feature was Hour (the time of day), followed by Number_of_casualties, Cause_of_accident, and Day_of_week. This does match real-world intuition, since accidents at certain hours (like late at night with poor visibility) and accidents that already involve more casualties or a more dangerous cause are naturally more likely to be severe.

Q3. How could a real traffic police department or city planning office use this feature importance information in practice?

A traffic department could focus extra patrolling, better street lighting, or awareness campaigns during the hours and days that the model flags as high-risk. City planners could also use factors like Area_accident_occured and Types_of_Junction to identify specific dangerous locations or junction designs that need safety improvements.

Q4. Feature importance in this project came from the Random Forest model. Do you think Logistic Regression can also give a similar kind of insight? (Hint: research the term "coefficients" in Logistic Regression.)

Yes, Logistic Regression can give similar insight through its coefficients. Each feature gets a coefficient value, and a larger coefficient (positive or negative) means that feature has a stronger influence on the prediction. However, since Logistic Regression assumes simpler, linear relationships, and our data has many categorical features, the Random Forest's feature importance is generally considered more reliable for this kind of dataset.

STEP 12: Make a Prediction on New (Sample) Data

One row was picked from the testing set and treated as a new accident report. The best model (Random Forest) predicted its severity, and the numeric prediction was converted back into a readable label using target_encoder.inverse_transform().

- Predicted Severity: Slight Injury
- Actual Severity: Slight Injury
- Result: Prediction is Correct

Q1. Why do we need to convert the model's numeric prediction back into a readable text label?

The model works internally with numbers (0, 1, 2) because that is what it was trained on. But these numbers have no meaning to a human reader on their own. Converting the prediction back into text labels like 'Slight Injury' makes the result understandable and useful for a real person, such as a police officer or city planner.

Q2. Was your model's prediction correct for the sample row you tested? If it was wrong, what could be a possible reason?

Yes, the model's prediction was correct — it predicted 'Slight Injury' for the sample row, and the actual severity was also 'Slight Injury'. If it had been wrong, a possible reason could be that this particular accident had an unusual combination of conditions that the model had not seen often enough during training, or that some of its casualty-related information was filled in with 'Unknown' or a median value rather than the real data.

Q3. In a real deployment (not just a classroom project), where do you think the "new" accident data would actually come from, instead of a row taken from your testing set?

In a real deployment, this data would likely come directly from a police officer or first responder entering accident details into a mobile app or system right at the accident scene (for example, weather, road type, number of vehicles), or from sensors and traffic cameras. The model would then instantly predict severity to help decide how urgently to respond.

STEP 13: Conclusion

Problem and Real-World Value:

This project solved the problem of predicting how severe a road accident is likely to be (Slight, Serious, or Fatal Injury) based on details like weather, road conditions, lighting, and number of vehicles involved. In the real world, this kind of model matters because it can help emergency services respond faster and more appropriately to accidents, and help city planners identify dangerous roads or junctions that need safety improvements.

Summary of Steps Followed:

- Set up the Python environment and loaded the required libraries.
- Loaded and explored the dataset (12,316 rows, 32 columns), and found strong class imbalance in Accident_severity.
- Cleaned the data by filling missing categorical values with 'Unknown' and missing numeric values with the median.
- Engineered a new Hour feature from the Time column, and removed columns that would cause data leakage.
- Encoded all categorical columns into numbers using Label Encoding.
- Split the data into 80% training and 20% testing sets, using stratify to preserve class balance.
- Trained three models — Logistic Regression, Decision Tree, and Random Forest — using class_weight='balanced'.
- Evaluated the models using accuracy, confusion matrix, and classification report, and found Random Forest to be the best model.
- Found the top features driving predictions using Random Forest's feature importance.
- Tested the final model on a new sample row and got a correct prediction.

Q1. What are two limitations of your model that you personally noticed while working on this project?

First, the dataset is heavily imbalanced — there are only 158 Fatal injury cases out of 12,316 records, so even with class_weight='balanced', the model still finds it hard to predict the rare Fatal injury class accurately, as shown by its low precision for that class. Second, a large number of missing values were filled using 'Unknown' or the median rather than real collected data (for example, over 4,000 missing values in Defect_of_vehicle), which may have slightly reduced the true accuracy of the model.

Q2. Suggest one realistic improvement that could help your model become more accurate or fair, especially for rare "Fatal" cases. (You may research terms like SMOTE, XGBoost, or GridSearchCV as starting points.)

One realistic improvement would be to use SMOTE (Synthetic Minority Oversampling Technique), which creates new, synthetic examples of the rare Fatal injury class so the model gets more balanced exposure to it during training. Using GridSearchCV to fine-tune the Random Forest's hyperparameters (like n_estimators and max_depth), or trying a more advanced model like XGBoost, could also help improve overall accuracy and fairness across all three classes.

Q3. In 3-4 sentences, summarize the real-world value and impact of this project, as if you were explaining it to a non-technical person.

This project builds a computer model that looks at the details of a road accident — like the weather, time of day, and number of vehicles involved — and predicts how serious the accident is likely to be. This can help emergency services decide how quickly and with what resources to respond to an accident. It can also help city authorities spot risky roads, junctions, or time periods so they can take steps like better lighting or road repairs to prevent accidents in the future. In short, it turns raw accident records into useful, life-saving insights.

Thank You